

Chương 9:

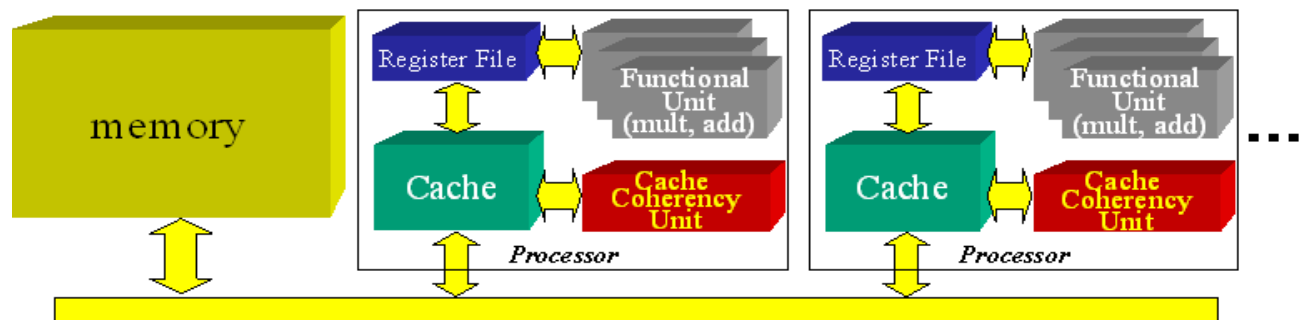
TỔ CHỨC MÁY TÍNH SONG SONG

NỘI DUNG

- › Xử lý song song
- › Máy tính đa nhân

Multiprocessor

Multiprocessor Architecture



Cache coherency unit will intervene if two or more processors attempt to update same cache line

- All memory (and I/O) is shared by all processors
- Read/write conflicts between processors on the same memory location are resolved by *cache coherency unit*
- Programming model is an extension of single processor programming model

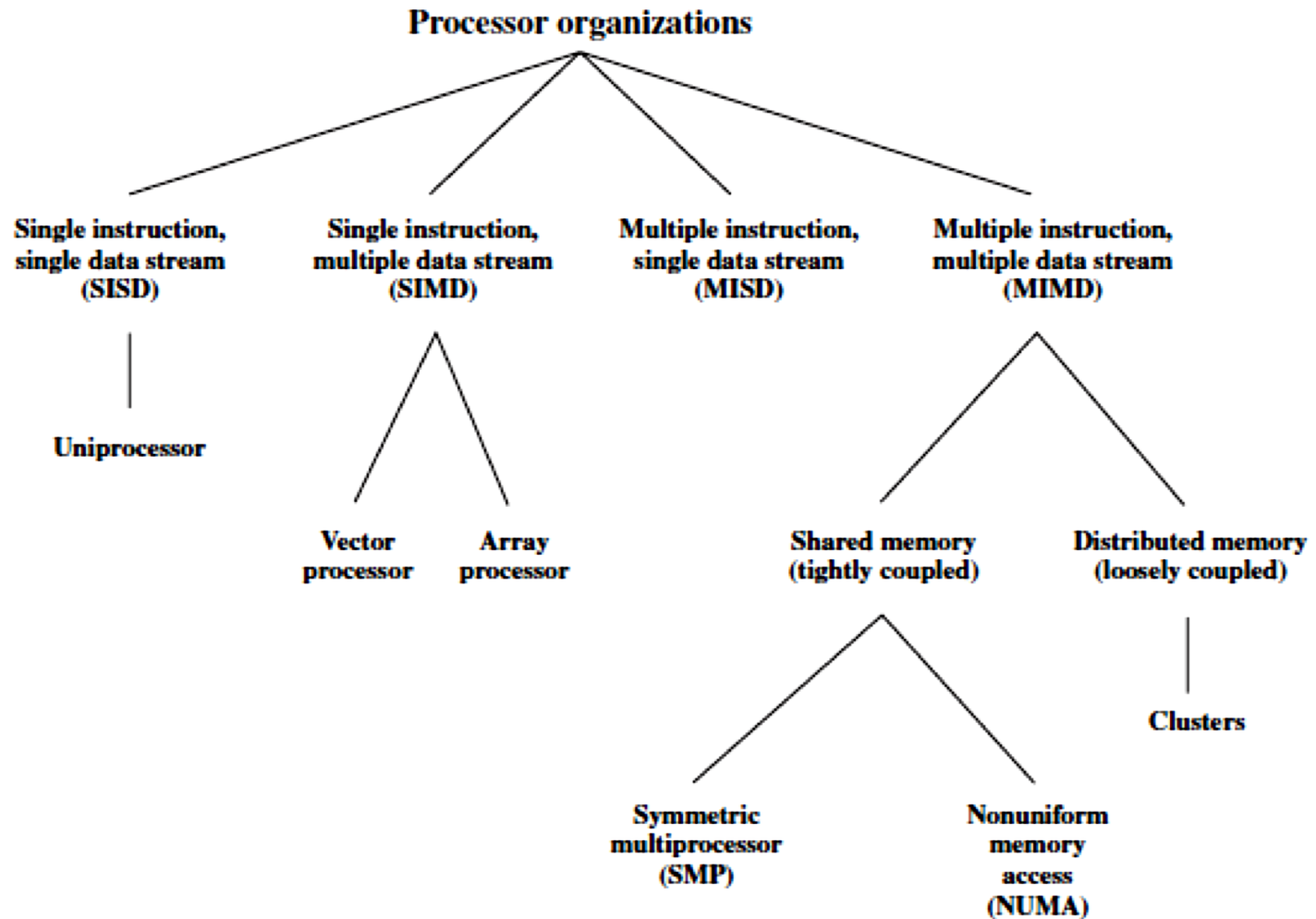
Multiprocessor

- › Đa vi xử lý (hoặc nhiều vi xử lý): một hệ thống máy tính với hai hoặc nhiều đơn vị xử lý trung tâm (CPU), mỗi đơn vị này chia sẻ bộ nhớ chính chung cũng như các thiết bị ngoại vi. Điều này giúp xử lý đồng thời các chương trình.
- › Mục tiêu chính của việc sử dụng Multiprocessor là để tăng tốc độ thực thi của hệ thống, khả năng chịu lỗi và phù hợp với ứng dụng.

Multiprocessor

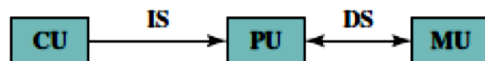
- › Multiprocessor được coi là một phương tiện để cải thiện tốc độ máy tính, hiệu quả và chi phí-hiệu quả, cũng như để cung cấp tăng cường tính sẵn sàng và độ tin cậy.
- › Trong một hệ thống Multiprocessor, tất cả các CPU có thể được bình đẳng, hoặc một số có thể được dành cho các mục đích riêng biệt.

Phân loại hệ thống máy tính

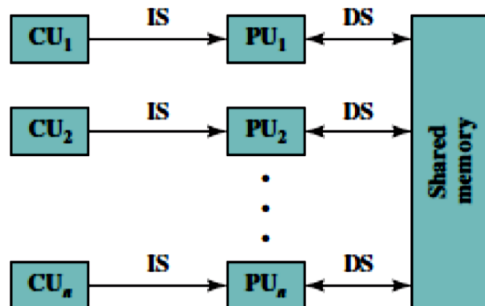


Phân loại hệ thống máy tính

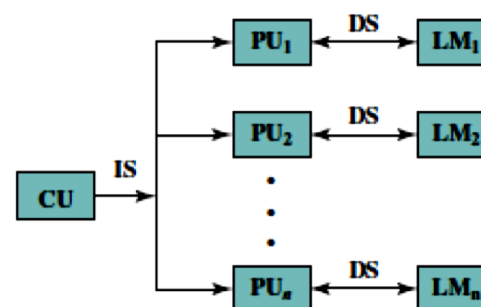
- › SISD - Single Instruction Stream, Single Data Stream
- › SIMD - Single Instruction Stream, Multiple Data Stream
- › MISD - Multiple Instruction Stream, Single Data Stream
- › MIMD



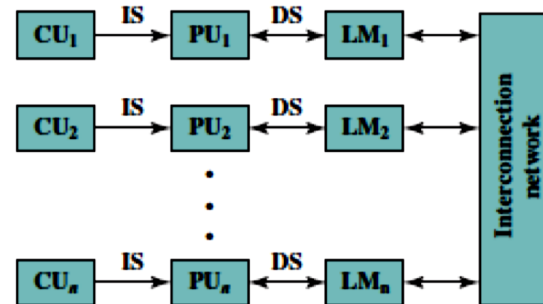
(a) SISD



(c) MIMD (with shared memory)



(b) SIMD (with distributed memory)



(d) MIMD (with distributed memory)

CU = Control unit SISD = Single instruction,
 IS = Instruction stream = single data stream
 PU = Processing unit SIMD = Single instruction,
 DS = Data stream multiple data stream
 MU = Memory unit MIMD = Multiple instruction,
 LM = Local memory multiple data stream

SISD

› Single instruction, single data (SISD) stream:

- Một bộ xử lý thực thi một luồng lệnh duy nhất để xử lý dữ liệu được lưu trữ trong một bộ nhớ.
- Chính là Kiến trúc von Neumann

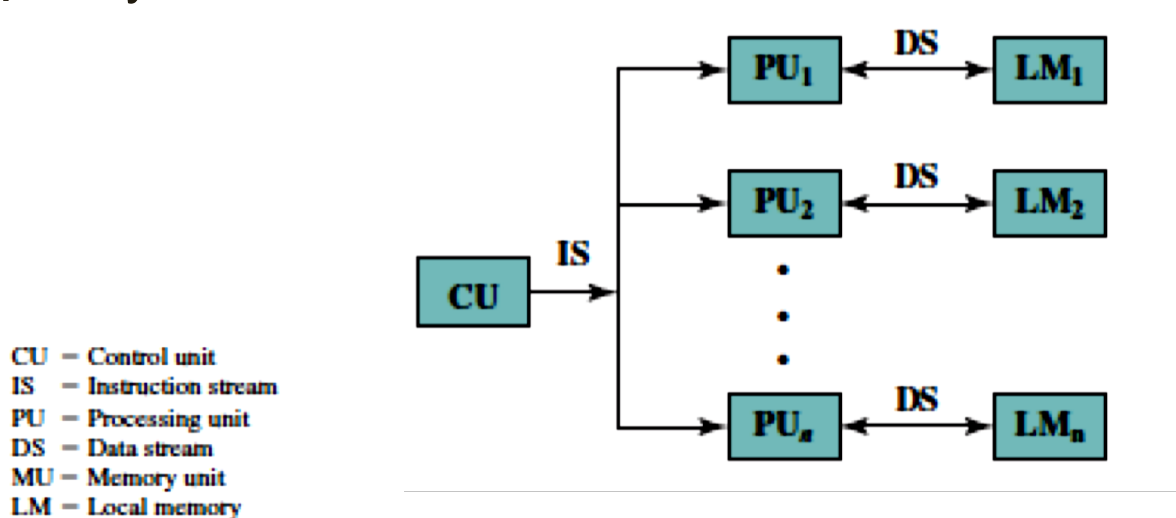


CU – Control unit
IS – Instruction stream
PU – Processing unit
DS – Data stream
MU – Memory unit
LM – Local memory

SIMD

› Single instruction, multiple data (SIMD) stream:

- Một lệnh máy điều khiển việc thực thi đồng thời một số phần tử dữ liệu cần xử lý.
- Mỗi phần tử dữ liệu này có liên quan một bộ nhớ dữ liệu
- Các lệnh được thực thi trên các bộ dữ liệu khác nhau bởi các bộ xử lý khác nhau.

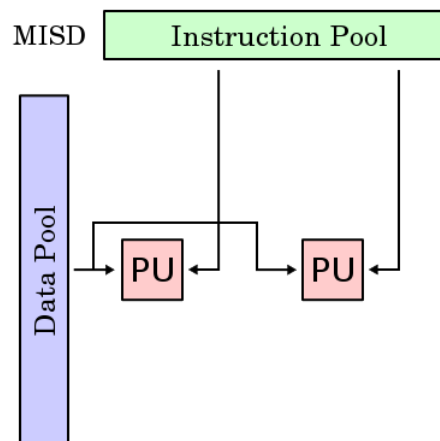


MISD

› Multiple instruction, single data (MISD) stream:

- Một chuỗi dữ liệu được truyền đến một bộ các bộ xử lý, mỗi bộ vi xử lý thực hiện một chuỗi lệnh khác nhau.
- Cấu trúc này không được thực hiện thương mại / Không tồn tại máy tính thực tế.

CU = Control unit
IS = Instruction stream
PU = Processing unit
DS = Data stream
MU = Memory unit
LM = Local memory

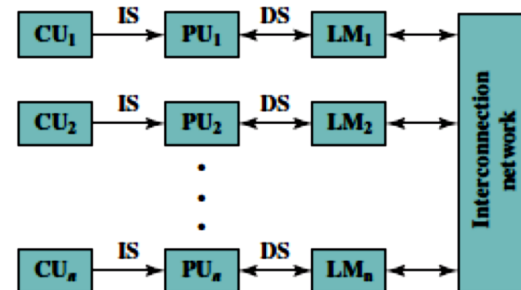
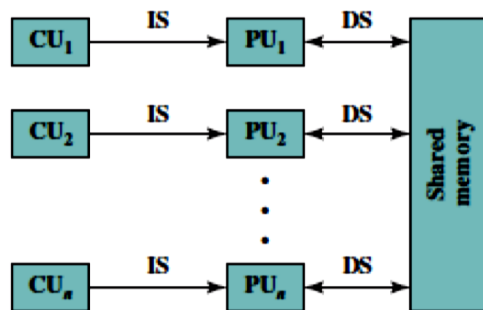


MIMD

› Multiple instruction, multiple data (MIMD) stream:

- Các bộ xử lý đồng thời thực hiện các dãy lệnh khác nhau trên các dữ liệu khác nhau
- Các mô hình MIMD
 - › Multiprocessors (Shared Memory): **symmetric multiprocessor (SMP), nonuniform memory access (NUMA)**
 - › Multicomputers (Distributed Memory): **cluster**

CU = Control unit
IS = Instruction stream
PU = Processing unit
DS = Data stream
MU = Memory unit
LM = Local memory



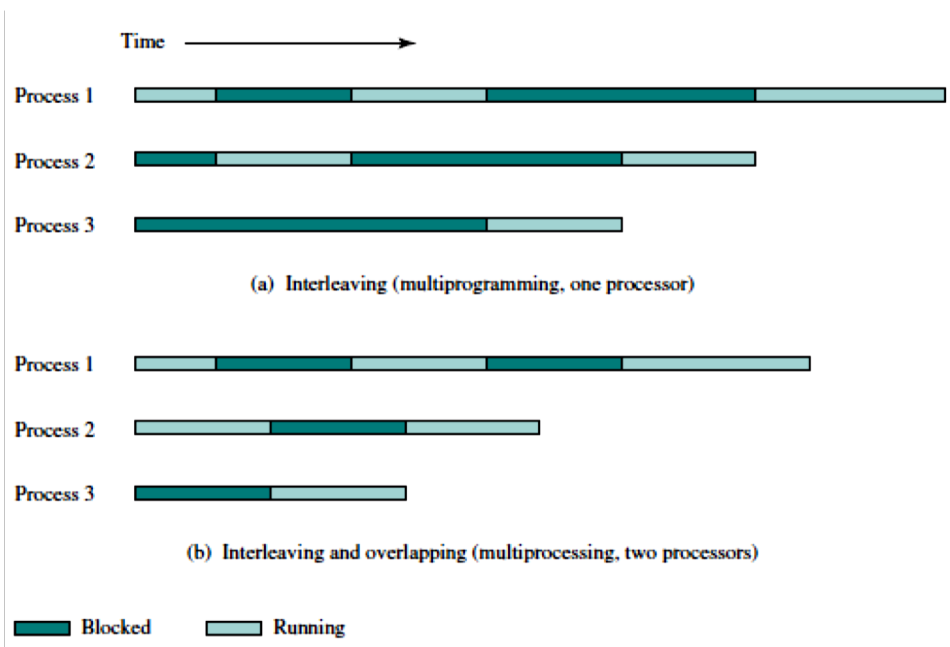
MIMD

› **Symmetric multiprocessor (SMP) :**

- Có hai hoặc nhiều bộ xử lý tương đương nhau.
- Các bộ xử lý này có chung bộ nhớ chính và các cổng I/O và được kết nối với nhau bằng BUS hoặc sơ đồ kết nối nội bộ khác, sao cho thời gian truy cập bộ nhớ là gần như nhau cho mỗi bộ xử lý.
- Tất cả các bộ xử lý chia sẻ quyền truy cập vào các thiết bị I/O, thông qua cùng một kênh hoặc thông qua các kênh khác nhau cung cấp đường dẫn đến cùng một thiết bị.
- Tất cả các bộ xử lý có thể thực hiện các chức năng giống nhau (do đó thuật ngữ đối xứng).
- Hệ thống được điều khiển bởi một hệ điều hành tích hợp hỗ trợ sự tương tác giữa bộ xử lý và các chương trình của nó.

SMP

- › Một tổ chức SMP có một số lợi thế so với một tổ chức chỉ có đơn bộ xử lý
 - **Hiệu suất:** một công việc được thực hiện bởi máy tính có thể được tổ chức sao cho một số phần của công việc có thể được thực hiện song song, thì một hệ thống có nhiều bộ xử lý sẽ mang lại hiệu suất cao hơn so với một bộ xử lý cùng loại

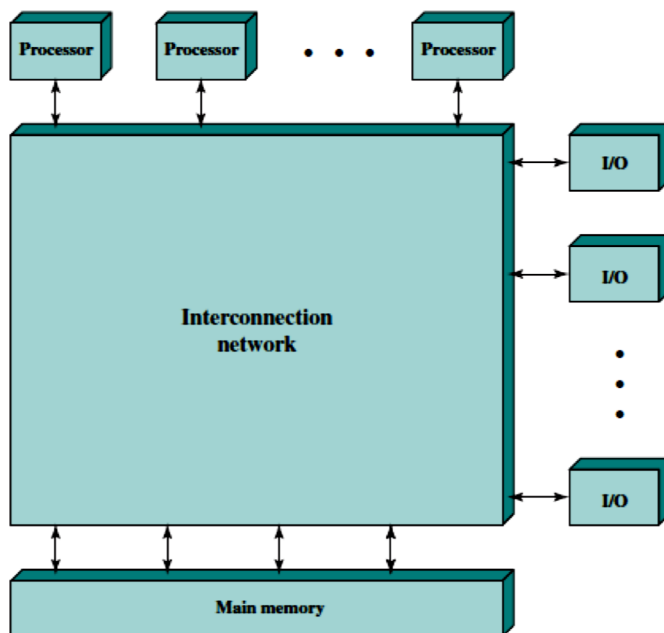


SMP

- › Một tổ chức SMP có một số lợi thế so với một tổ chức chỉ có đơn bộ xử lý
 - **Tính khả dụng:** tất cả các bộ xử lý có thể thực hiện các chức năng giống nhau, sự hỏng hóc của một bộ xử lý sẽ không làm dừng máy. Thay vào đó, hệ thống có thể tiếp tục hoạt động với hiệu suất giảm hơn.
 - **Tăng trưởng tăng dần:** Người dùng có thể nâng cao hiệu suất của hệ thống bằng cách thêm bộ vi xử lý bổ sung. Các nhà cung cấp có thể cung cấp một loạt các sản phẩm với các đặc tính giá cả và hiệu suất khác nhau dựa trên số lượng bộ xử lý được định cấu hình trong hệ thống.

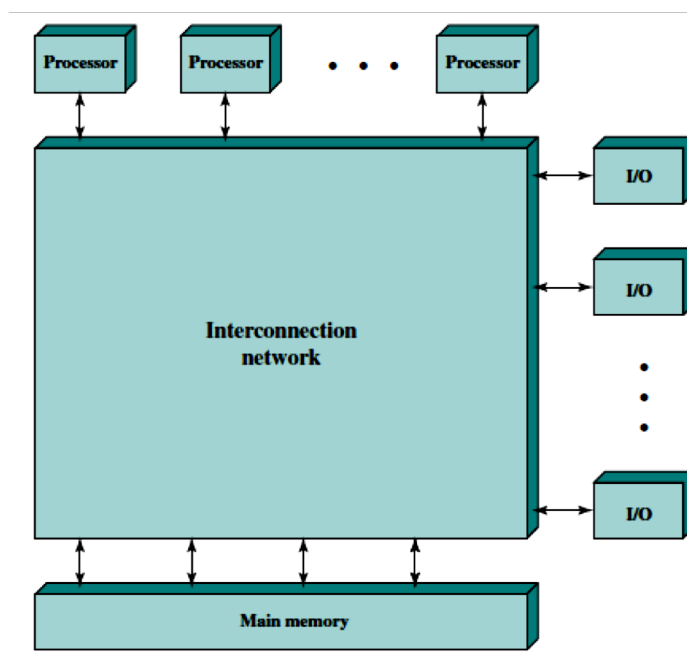
SMP

- › Một hệ thống đa bộ vi xử lý: Có hai hoặc nhiều bộ vi xử lý.
- › Mỗi bộ xử lý đều khép kín, bao gồm một đơn vị điều khiển, ALU, các thanh ghi và thông thường, một hoặc nhiều mức bộ đệm.
- › Mỗi bộ xử lý có quyền truy cập vào bộ nhớ chính được chia sẻ và các thiết bị I/O thông qua một số dạng cơ chế kết nối.

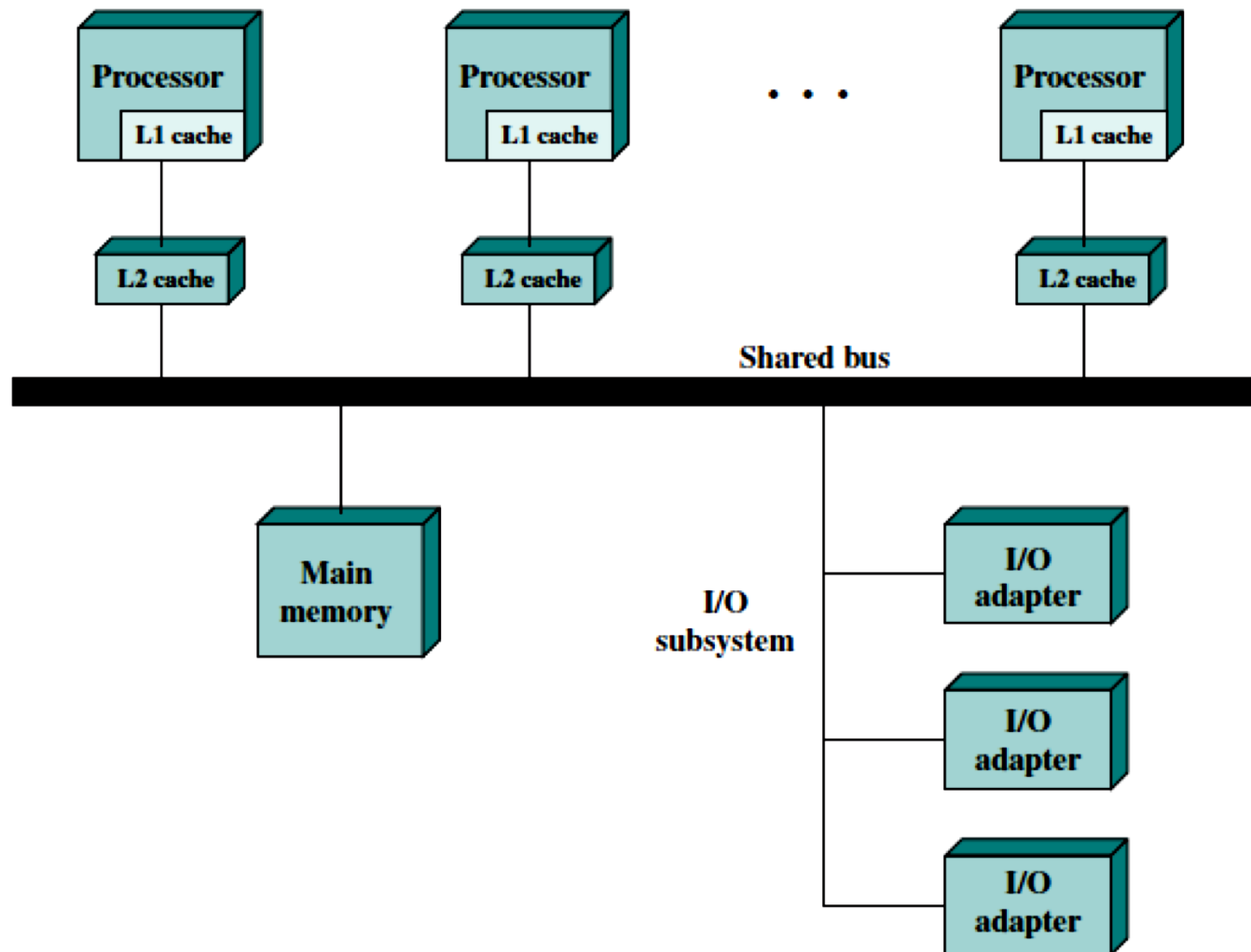


SMP

- › Các bộ vi xử lý có thể giao tiếp với nhau thông qua bộ nhớ (messages và thông tin trạng thái còn lại trong các vùng dữ liệu chung), các bộ vi xử lý có thể trao đổi tín hiệu trực tiếp.
- › Bộ nhớ thường được tổ chức sao cho có thể truy cập đồng thời nhiều khối bộ nhớ riêng biệt. Trong một số cấu hình, mỗi bộ xử lý cũng có thể có bộ nhớ chính và kênh I/O riêng ngoài các tài nguyên được chia sẻ.



SMP



NUMA

› **Uniform memory access (UMA)**

- Tất cả các bộ xử lý có quyền truy cập vào tất cả các phần của bộ nhớ chính bằng cách sử dụng LOAD/STORE.
- Thời gian truy cập vào tất cả các vùng của bộ nhớ là như nhau.
- Thời gian truy cập vào bộ nhớ cho các bộ xử lý khác nhau là như nhau.

› **Truy cập bộ nhớ nonuniform (NUMA)**

- Tất cả các bộ xử lý có quyền truy cập vào tất cả các phần của bộ nhớ chính bằng cách sử dụng LOAD/STORE .
- Thời gian truy cập của bộ xử lý khác nhau tùy thuộc vào vùng bộ nhớ chính đang được truy cập.
- Bộ xử lý khác nhau truy cập các vùng bộ nhớ khác nhau ở tốc độ khác nhau (ùng nhớ nào chậm hơn, vùng nào khác nhau nhanh hơn...).

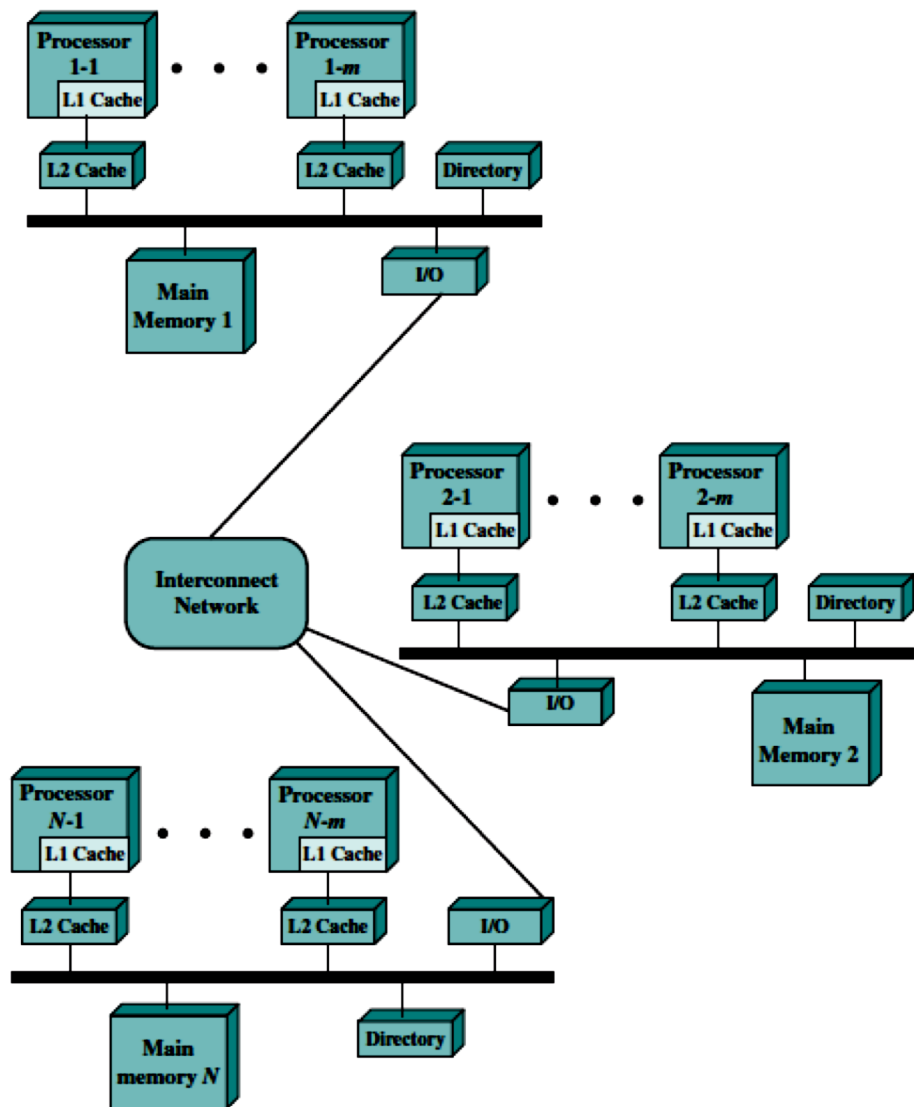
› **Cache-coherent NUMA (CC-NUMA)**

- Một hệ thống NUMA trong đó sự gắn kết bộ nhớ cache được duy trì giữa các bộ đệm của các bộ xử lý khác nhau.

NUMA

- › **SMP:** có giới hạn thực tế đối với số lượng bộ xử lý có thể được sử dụng
 - Giới hạn lưu lượng BUS đối với khoảng từ 16 đến 64 bộ xử lý.
- › **CLUSTER:** Trong các cụm, mỗi nút có bộ nhớ chính riêng
 - Các ứng dụng không nhận biết được bộ nhớ toàn cục
 - Sự kết hợp được duy trì bởi phần mềm chứ không phải phần cứng.
- › **NUMA:** giữ lại thuộc tính SMP trong khi cung cấp đa xử lý quy mô lớn.
 - Mục tiêu với NUMA là duy trì bộ nhớ trong suốt toàn hệ thống trong khi cho phép nhiều nút đa bộ vi xử lý, mỗi nút có BUS riêng hoặc hệ thống kết nối nội bộ.

CC-NUMA



NUMA

- › Hệ thống CC-NUMA là nó có thể mang lại hiệu suất thực hiện hiệu quả ở mức độ song song cao hơn SMP mà không yêu cầu thay đổi phần mềm chính.
- › Lưu lượng BUS trên bất kỳ nút riêng lẻ bị giới hạn theo yêu cầu mà BUS có thể xử lý.
- › Nếu nhiều truy cập bộ nhớ đến các nút từ xa, hiệu suất bắt đầu mất hiệu quả.
- › Thay đổi phần mềm sẽ được yêu cầu để chuyển hệ điều hành và ứng dụng từ SMP sang hệ thống CC-NUMA.

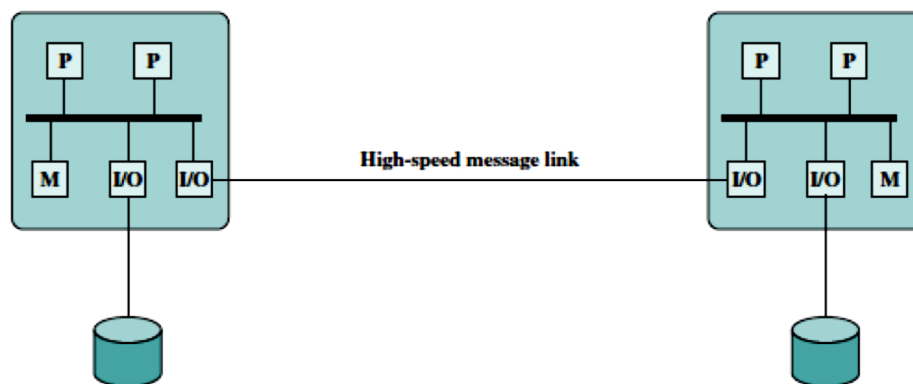
CLUSTER

- › Các máy tính trong một cụm có chia sẻ quyền truy cập vào cùng một đĩa nhớ được chia sẻ dung chung (hoặc không):
 - Những CPU nằm trong nhiều máy tính giống nhau, lân cận nhau (gọi là các node, node card hay computer node) và chúng được kết nối nhằm tạo ra một hệ thống lớn hơn, hoàn chỉnh hơn.
 - Người ta xem cả hệ thống như một siêu máy tính duy nhất. Với biện pháp này, các nhà thiết kế cần đảm bảo rằng tốc độ cũng như tính linh hoạt nội liên kết giữa các máy tính phải đủ đáp ứng yêu cầu công việc.
 - Theo số liệu từ TOP500, số siêu máy tính cluster hiện chiếm đến 82,2% thị phần siêu máy tính toàn cầu. Siêu máy tính IBM Blue Gene/Q sử dụng dạng cluster.

CLUSTER

a) Một cụm hai nút trong đó liên kết duy nhất là bằng một liên kết tốc độ cao có thể được sử dụng để trao đổi thông tin để phối hợp hoạt động của cụm.

- Liên kết có thể là một mạng LAN được chia sẻ với các máy tính khác không phải là một phần của cụm hoặc liên kết có thể là một kết nối chuyên dụng.

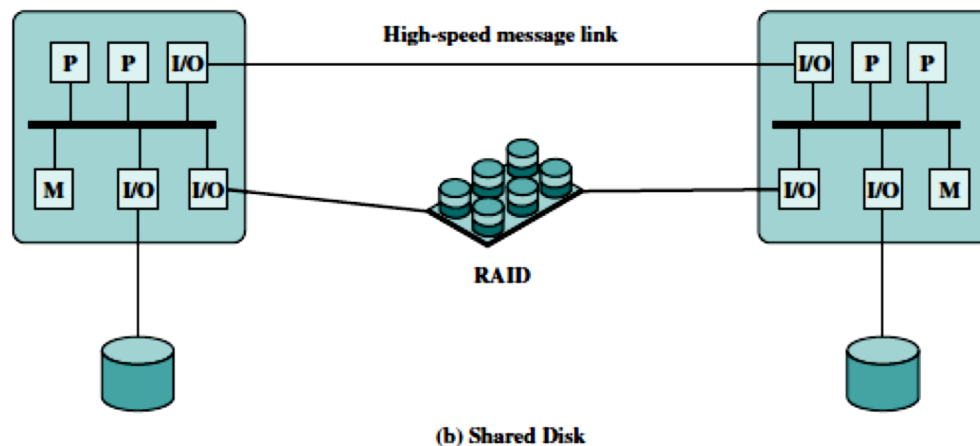


(a) Standby server with no shared disk

CLUSTER

b) Trang bị thêm cụm đĩa nhớ chia sẻ dung chung.

- Đĩa được liên kết trực tiếp với nhiều máy tính trong cụm. Hệ thống con đĩa chung là hệ thống RAID.
- Việc sử dụng RAID hoặc một số công nghệ đĩa dự phòng khác là phổ biến trong các cụm sao cho tính khả dụng cao đạt được bởi sự hiện diện của nhiều máy tính không bị ảnh hưởng bởi một ổ đĩa chung.



Tính toán song song

- › Ba cách tiếp cận chung cho vấn đề:
 - Trình biên dịch song song (**Parallelizing compiler**).
 - Ứng dụng song song (**Parallelized application**).
 - Tính toán tham số (**Parametric computing**).

Tính toán song song

› Trình biên dịch song song (**Parallelizing compiler**):

- Tại thời điểm biên dịch, phần nào của ứng dụng có thể được thực thi song song.
- Các phần này sau đó được tách ra để được gán cho các máy tính khác nhau trong cụm.
- Hiệu suất phụ thuộc vào bản chất của vấn đề và trình biên dịch được thiết kế tốt như thế nào. *Nói chung, trình biên dịch như vậy rất khó để phát triển.*

Tính toán song song

- › Ứng dụng song song (**Parallelized application**):
 - Lập trình viên viết ứng dụng ngay từ đầu để chạy trên một cụm và sử dụng thông điệp truyền để di chuyển dữ liệu, theo yêu cầu, giữa các nút cụm.
 - Điều này đặt một thách thức lớn lên lập trình viên nhưng *có thể là cách tiếp cận tốt nhất để khai thác các cụm cho một số ứng dụng.*

Tính toán song song

- › Tính toán tham số (**Parametric computing**):
 - Cách tiếp cận này có thể được sử dụng nếu bản chất của ứng dụng là một thuật toán hoặc chương trình phải được thực thi với số lượng lớn, mỗi lần với một tập hợp các điều kiện hoặc tham số bắt đầu khác nhau.
 - Một ví dụ điển hình là một mô hình mô phỏng, sẽ chạy một số lượng lớn các kịch bản khác nhau và sau đó phát triển các bản tóm tắt thống kê về kết quả.
 - Để phương pháp này có hiệu quả, *cần có các công cụ xử lý tham số để tổ chức, điều hành và quản lý công việc một cách hiệu quả.*

MULTITHREADING AND CHIP MULTIPROCESSORS

- › **Hiệu suất** của bộ xử lý có thể được đo bằng tốc độ thực thi các lệnh
 - ***MIPS rate = f * IPC***
 - › f = tần số xung nhịp của bộ xử lý, tính bằng MHz
 - › IPC = Số lệnh trung bình trên mỗi chu kỳ
 - Tăng hiệu suất bằng cách tăng tần số xung nhịp và tăng số lệnh hoàn thành trong chu kỳ
- › **Multithreading:**
 - Cho phép mức độ song song cao ở mức lệnh mà không làm tăng độ phức tạp của mạch hoặc mức tiêu thụ điện.
 - Luồng lệnh được chia thành nhiều luồng nhỏ hơn và có thể được thực thi song song

Implicit and Explicit Multithreading

- › Tất cả các bộ xử lý thương mại và hầu hết các bộ xử lý thử nghiệm cho đến nay đều sử dụng đa luồng rõ ràng (**Explicit Multithreading**).
 - Các hệ thống này đồng thời thực hiện các lệnh từ các luồng rõ ràng khác nhau, bằng cách xen kẽ các lệnh từ các luồng khác nhau trên các đường ống dùng chung (shared pipelines) hoặc bằng cách thực hiện song song trên các đường ống song song.
- › Đa luồng tiềm ẩn đề (**Implicit Multithreading**) cập đến việc thực thi đồng thời nhiều luồng được trích xuất từ một chương trình tuần tự duy nhất.
 - Các luồng ngầm định này có thể được định nghĩa tĩnh bởi trình biên dịch hoặc động bởi phần cứng.

Explicit Multithreading

- › Bốn cách tiếp cận chính đối với đa luồng:
 - **Interleaved**
 - **Blocked**
 - **Simultaneous**
 - **Chip**

Cách tiếp cận để thực hiện đa luồng

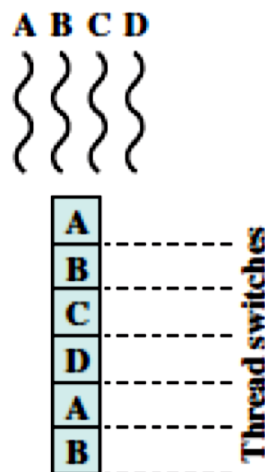
- › **Single-threaded scalar**: kỹ thuật đường ống đơn giản được tìm thấy trong các máy RISC và CISC truyền thống, không có đa luồng.



Cách tiếp cận để thực hiện đa luồng

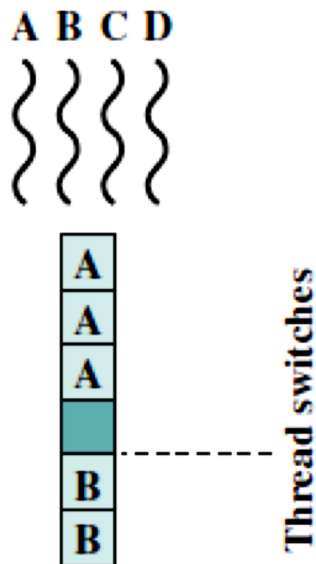
› **Interleaved multithreading scalar**(Đa luồng xen kẽ): đa luồng mịn

- Bộ vi xử lý xử lý hai hoặc nhiều bối cảnh luồng tại một thời điểm, chuyển từ luồng này sang luồng khác ở mỗi chu kỳ xung nhịp.
- Nếu một luồng bị chặn do phụ thuộc dữ liệu hoặc độ trễ bộ nhớ, luồng đó bị bỏ qua và luồng sẵn sàng được thực thi.



Cách tiếp cận để thực hiện đa luồng

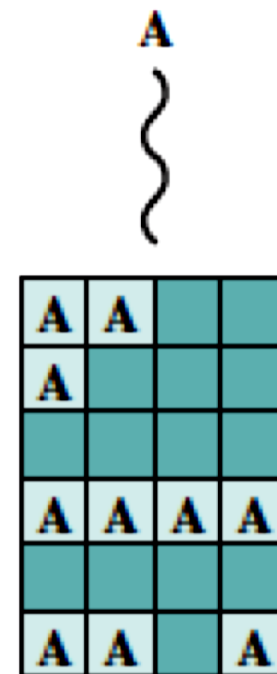
- › **Blocked multithreading scalar** (Đa luồng bị chặn): đa luồng thô
 - Các lệnh của một luồng được thực hiện liên tiếp cho đến khi một sự kiện xảy ra có thể gây ra sự chậm trễ và sẽ dừng pipeline, chẳng hạn như cache miss.



Cách tiếp cận để thực hiện đa luồng

› Superscalar:

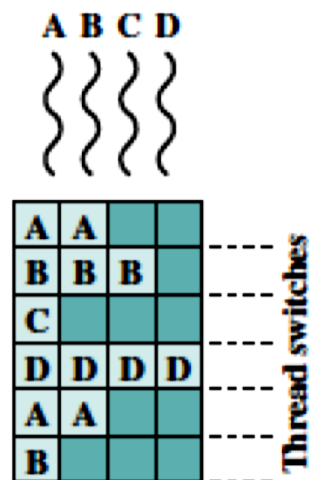
- Đây là cách tiếp cận không có đa luồng. Cho đến gần đây, đây là cách tiếp cận mạnh mẽ nhất để cung cấp tính song song trong bộ xử lý.
- **Lưu ý** rằng trong một số chu kỳ, không phải tất cả các vị trí có sẵn đều được sử dụng.
 - › Trong các chu kỳ này, không phải tất cả các lệnh được cấp phát, điều này được gọi là mất ngang (**horizontal loss**).
 - › Trong các chu kỳ lệnh khác, không có vị trí nào được sử dụng; đây là những chu kỳ khi không có lệnh nào có thể được ban hành; điều này được gọi là mất dọc (**vertical loss**).



Cách tiếp cận để thực hiện đa luồng

› Interleaved multithreading superscalar:

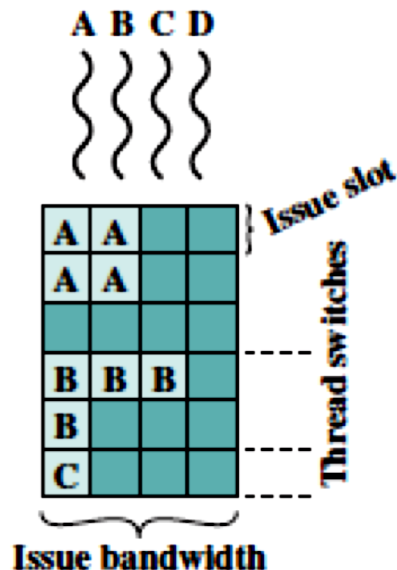
- Trong mỗi chu kỳ, càng nhiều lệnh được phát ra từ một luồng càng tốt . Với kỹ thuật này, độ trễ tiềm ẩn do chuyển mạch luồng được loại bỏ.
- Tuy nhiên, số lượng lệnh được ban hành trong bất kỳ chu kỳ nào vẫn bị giới hạn bởi các phụ thuộc tồn tại bên trong bất kỳ luồng đã cho.



Cách tiếp cận để thực hiện đa luồng

› Blocked multithreaded superscalar:

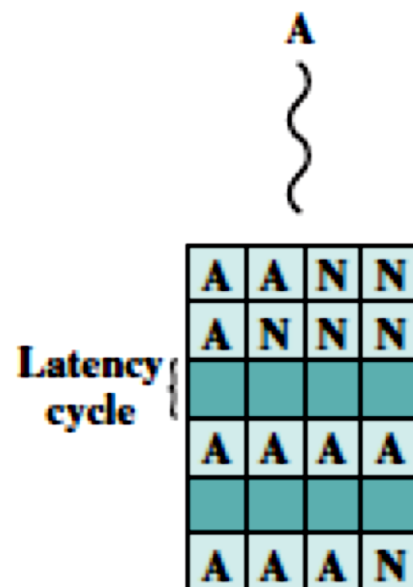
- Các lệnh từ chỉ một luồng có thể được ban hành trong bất kỳ chu kỳ nào và đa luồng bị chặn được sử dụng.



Cách tiếp cận để thực hiện đa luồng

› Very long instruction word (VLIW):

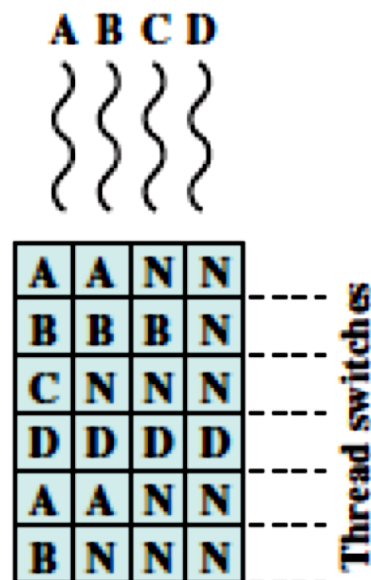
- Kiến trúc VLIW, chẳng hạn như IA-64, đặt nhiều lệnh trong một từ duy nhất.
- Thông thường, một VLIW được xây dựng bởi trình biên dịch, nơi đặt các hoạt động có thể được thực thi song song trong cùng một từ.
- Trong một máy VLIW đơn giản, nếu không thể điền hoàn toàn từ đó bằng các lệnh được phát song song, NO-OPS được sử dụng.



Cách tiếp cận để thực hiện đa luồng

› Interleaved multithreading VLIW:

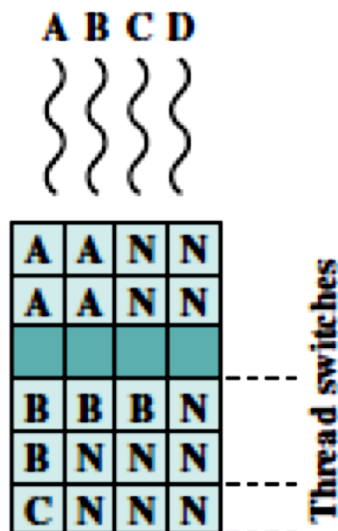
- Cách tiếp cận này sẽ cung cấp hiệu quả tương tự với những phương pháp được cung cấp bởi đa luồng xen kẽ (**interleaved multithreading**) trên một kiến trúc **superscalar**.



Cách tiếp cận để thực hiện đa luồng

› Blocked multithreaded VLIW:

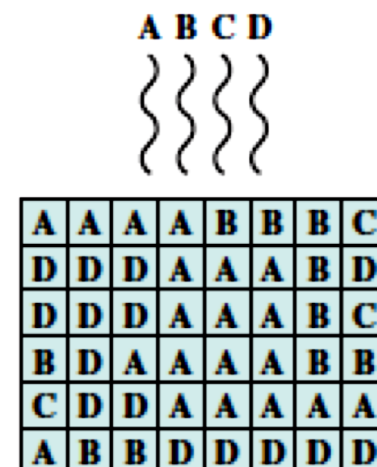
- Cách tiếp cận này sẽ cung cấp hiệu quả tương tự với các phương pháp được cung cấp bởi đa luồng bị chặn (**blocked multithreading**) trên một kiến trúc superscalar



Cách tiếp cận để thực hiện đa luồng

› Simultaneous multithreading:

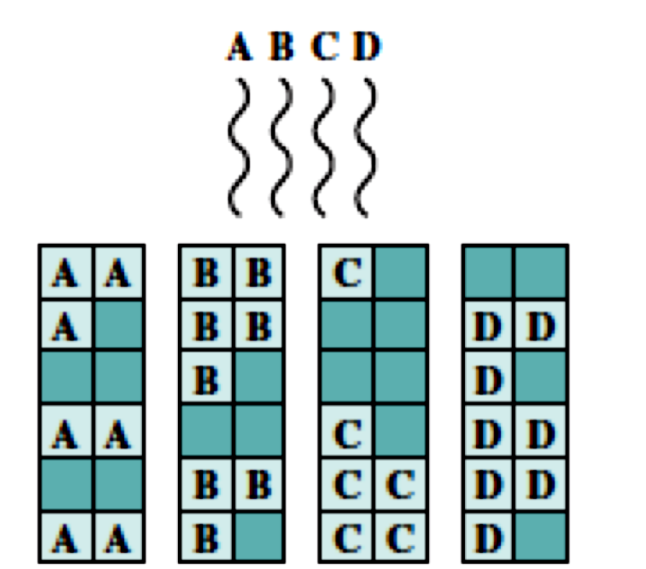
- Một hệ thống có khả năng ban hành 8 lệnh cùng một lúc (minh họa).
- Nếu một luồng có mức độ lệnh song song cao, thì trong một số chu kỳ, nó có thể lấp đầy tất cả các khe ngang.
- Trong các chu kỳ khác, các lệnh từ hai hoặc nhiều luồng có thể được cấp phát, thông thường có thể đưa ra số lượng lệnh tối đa trên mỗi chu kỳ nhằm cung cấp mức độ hiệu quả cao.



Cách tiếp cận để thực hiện đa luồng

› Chip multiprocessor (multicore):

- Hình minh họa cho thấy một con chip chứa bốn bộ xử lý, mỗi bộ xử lý có hai **superscalar**.
- Mỗi bộ xử lý được gán một luồng, từ đó nó có thể đưa ra tối đa hai lệnh cho mỗi chu kỳ.



Cách tiếp cận để thực hiện đa luồng

› Pentium 4

- Pentium 4 gần đây sử dụng kỹ thuật đa luồng, gọi là siêu phân luồng *hyperthreading*.
- Về bản chất, cách tiếp cận Pentium 4 là sử dụng **SMT** với sự hỗ trợ cho hai luồng.

Cách tiếp cận để thực hiện đa luồng

› **BM Power5**

- Được sử dụng trong các sản phẩm PowerPC cao cấp, kết hợp **chip multiprocessing** với **SMT**.
- Chip này có hai bộ xử lý riêng biệt, mỗi bộ xử lý là đa luồng xử lý (multithreaded processor) hỗ trợ hai luồng đồng thời sử dụng SMT.
- Việc có hai bộ xử lý SMT hai chiều trên một chip duy nhất mang lại hiệu năng vượt trội cho bộ xử lý SMT bốn chiều.

Vector Computation

- › Mặc dù hiệu suất của các máy tính lớn (**mainframe computers**) tiếp tục cải thiện không ngừng, nhưng vẫn có những ứng dụng nằm ngoài tầm với :
 - Cần có máy tính chuyên biệt để giải quyết các vấn đề toán học của các quá trình vật lý: khí động học, địa chấn, khí tượng học, và vật lý nguyên tử, hạt nhân và plasma.

Vector Computation

- › Sự cần thiết phải có độ chính xác cao và một chương trình thực hiện lặp lại các phép toán số học dấu phẩy động trên các mảng số lớn.
 - Một trạng thái vật lý có thể được mô tả bởi một bề mặt ba chiều được xấp xỉ bởi một lưới các điểm.
 - Một tập hợp các phương trình vi phân xác định tính chất của bề mặt tại mỗi điểm dưới dạng một mảng các giá trị và hệ số, và giải pháp liên quan đến các phép toán số học lặp đi lặp lại trên các mảng dữ liệu.

Vector Computation

- › Siêu máy tính được phát triển để xử lý các loại vấn đề này :
 - Có khả năng hoạt động hàng tỷ điểm tính toán mỗi giây và được tối ưu hóa cho kiểu tính toán số vừa mô tả.
 - Các siêu máy tính đã sử dụng hạn chế và, vì giá cả và thị trường hạn chế.
 - Một vài trong số các máy này đang hoạt động, chủ yếu tại các trung tâm nghiên cứu và một số cơ quan chính phủ có chức năng khoa học hoặc kỹ thuật. Cũng như các lĩnh vực khác của công nghệ máy tính, luôn có nhu cầu tăng hiệu năng của siêu máy tính..

Vector Computation

- › Một loại hệ thống khác đã được thiết kế để giải quyết nhu cầu tính toán véc tơ, được gọi là bộ xử lý mảng (array processor).
 - Mặc dù siêu máy tính được tối ưu hóa cho tính toán véc tơ, nhưng nó là một máy tính có mục đích chung, có khả năng xử lý các tác vụ xử lý vô hướng và xử lý dữ liệu chung.
 - Bộ xử lý mảng không bao gồm xử lý vô hướng; chúng được cấu hình như các thiết bị ngoại vi bởi cả người dùng máy tính lớn và máy tính mini để chạy các phần chương trình được vector hóa.

Vector Computation

- › Mục đích để thiết kế bộ siêu máy tính hoặc bộ xử lý mảng:
 - Nhiệm vụ chính là thực hiện các phép toán số học trên các mảng hoặc vector của các số có dấu phẩy động.
 - Trong một máy tính có mục đích chung, điều này sẽ yêu cầu lập trình qua từng phần tử của mảng.

Vector Computation

- › Ví dụ, xét tổng hai vector (mảng một chiều) A và B, đặt kết quả vào C đòi hỏi sáu phép cộng riêng biệt.
 - Làm thế nào chúng ta có thể tăng tốc tính toán này khi thực hiện phép nhân $C=A \times B$ (mảng có kích thước là $N \times N$)?

$$\begin{array}{ccc} \left[\begin{array}{c} 1.5 \\ 7.1 \\ 6.9 \\ 100.5 \\ 0 \\ 59.7 \end{array} \right] & + & \left[\begin{array}{c} 2.0 \\ 39.7 \\ 1000.003 \\ 11 \\ 21.1 \\ 19.7 \end{array} \right] = \left[\begin{array}{c} 3.5 \\ 46.8 \\ 1006.093 \\ 111.5 \\ 21.1 \\ 79.4 \end{array} \right] \\ A & + & B = C \end{array}$$

$$c_{i,j} = \sum_{k=1}^N a_{i,k} \times b_{k,j}$$

Vector Computation

- › Vector processing.
- › Parallel processing.

```
DO 100 I = 1, N
DO 100 J = 1, N
C(I, J) = 0.0
DO 100 K = 1, N
C(I, J) = C(I, J) + A(I, K) + B(K, J)
100 CONTINUE
```

(a) Scalar processing

```
DO 100 I = 1, N
C(I, J) = 0.0 (J = 1, N)
DO 100 K = 1, N
C(I, J) = C(I, J) + A(I, K) + B(K, J) (J = 1, N)
100 CONTINUE
```

(b) Vector processing

```
DO 50 J = 1, N - 1
FORK 100
50 CONTINUE
J = N
100 DO 200 I = 1, N
C(I, J) = 0.0
DO 200 K = 1, N
C(I, J) = C(I, J) + A(I, K) + B(K, J)
200 CONTINUE
JOIN N
```

(c) Parallel processing

Vector Computation

› Vector processing:

- Điều này giả định rằng có thể tính toán trên một vector dữ liệu một chiều.
- Ký hiệu (**$J = 1 \dots N$**) chỉ ra rằng các phép tính trên tất cả các chỉ số **J** trong khoảng đã cho sẽ được thực hiện như một phép tính đơn lẻ.

```
DO 100 I = 1, N  
  C(I, J) = 0.0 (J = 1, N)  
  DO 100 K = 1, N  
    C(I, J) = C(I, J) + A(I, K) + B(K, J) (J = 1, N)  
100  CONTINUE
```

(b) Vector processing

Vector Computation

› Vector processing:

- Tất cả các phần tử của hàng thứ i sẽ được tính song song. Mỗi phần tử trong hàng là một tổng, và các tổng (trên K) được thực hiện một cách tuần tự thay vì song song.
- Chỉ có N^2 phép nhân vector được yêu cầu cho thuật toán này so với N^3 phép nhân vô hướng cho thuật toán vô hướng.

```
DO 100 I = 1, N  
  C(I, J) = 0.0 (J = 1, N)  
  DO 100 K = 1, N  
    C(I, J) = C(I, J) + A(I, K) + B(K, J) (J = 1, N)  
100 CONTINUE
```

(b) Vector processing

Vector Computation

› Parallel processing:

- Giả định rằng chúng ta có N bộ xử lý độc lập có thể hoạt động song song.
- Trong chương trình xử lý song song, mỗi cột của **C** được tính theo một quy trình riêng. Do đó, các phần tử trong một hàng **C** đã cho được tính song song.

```
DO 50 J = 1, N - 1  
  FORK 100  
50  CONTINUE  
    J = N  
100  DO 200 I = 1, N  
      C(I, J) = 0.0  
      DO 200 K = 1, N  
        C(I, J) = C(I, J) + A(I, K) + B(K, J)  
200  CONTINUE  
    JOIN N
```

(c) Parallel processing

Vector Computation

› Parallel processing:

- Lệnh **FORK** n dẫn đến tiến trình độc lập được bắt đầu tại vị trí n .
- Trong khi đó, tiến trình gốc tiếp tục thực thi lệnh ngay sau **FORK**. Mỗi lần thực hiện **FORK** đều tạo ra một quy trình mới. Lệnh **JOIN** về cơ bản là nghịch đảo của **FORK**.

```
DO 50 J = 1, N - 1
  FORK 100
50  CONTINUE
   J = N
100  DO 200 I = 1, N
      C(I, J) = 0.0
      DO 200 K = 1, N
        C(I, J) = C(I, J) + A(I, K) + B(K, J)
200  CONTINUE
     JOIN N
```

(c) Parallel processing

Vector Computation

› Parallel processing:

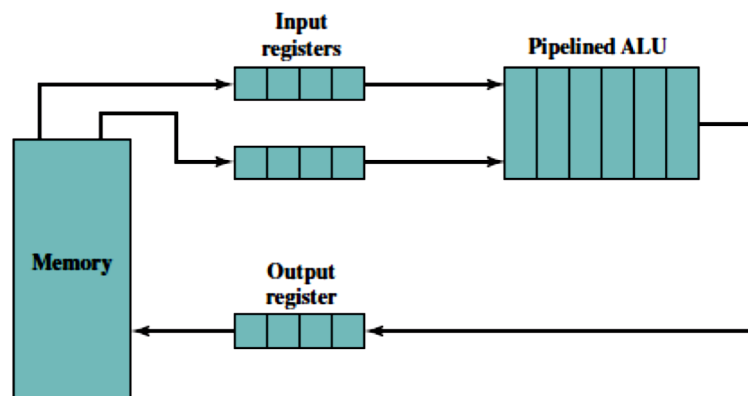
- Câu lệnh **JOIN N** dẫn đến **N** tiến trình độc lập để được hợp nhất thành một tiến trình tiếp tục thực hiện các lệnh theo sau **JOIN**.
- Hệ điều hành phải phối hợp việc hợp nhất này và do đó việc thực thi không tiếp tục cho đến khi tất cả các tiến trình **N** đã đi đến lệnh **JOIN**.

```
DO 50 J = 1, N - 1
  FORK 100
50  CONTINUE
    J = N
100  DO 200 I = 1, N
      C(I, J) = 0.0
      DO 200 K = 1, N
        C(I, J) = C(I, J) + A(I, K) + B(K, J)
200  CONTINUE
      JOIN N
```

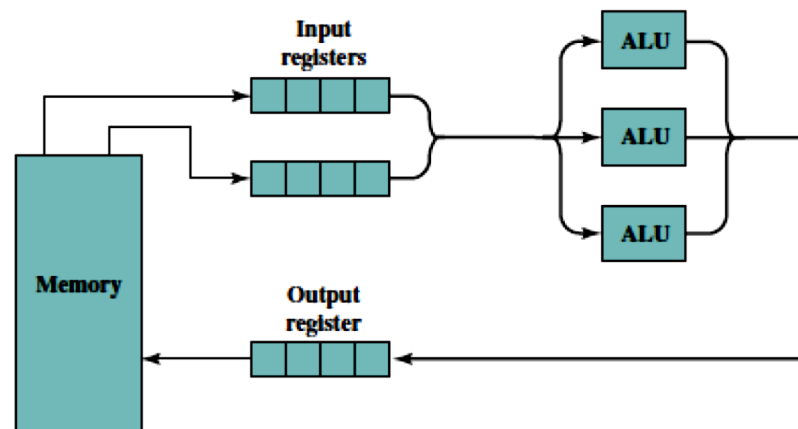
(c) Parallel processing

Vector Computation

- › Các loại tổ chức bộ xử lý có thể được sử dụng để thực hiện các phương pháp này:
 - **Pipelined ALU**
 - **Parallel ALUs**
 - **Parallel processors**



(a) Pipelined ALU



(b) Parallel ALUs

Vector Computation

